Lab04: A bounded model-and-tool loop

2026-10-21 · 80 minutes · Formative individual lab

Attend your registered Wednesday section: LAB 02 13:00–14:20 or LAB 01 17:30–18:50, LAS 1006. This lab has no separate course-weight mark; it prepares you for the published lab tests and individual project. Complete your own files. Discussion of concepts is allowed; do not exchange completed assessed project solutions.

Preparation

Prerequisite: October 19 architecture and model/tool interaction. TDD from the same Wednesday is not required. This lab is a smaller in-memory precursor to the project runtime.

Bring this folder and use JDK 21. The TA should have cached JUnit 6.0.3 before the session. Open `starter/` in VS Code, inspect `src/Main.java`, and run `bash run.sh compile`. The starter compiles but deliberately contains unfinished or incorrect exercise behaviour. Its check command is expected to fail until the tasks are completed; this is not an installation failure. No live model or API account is used here. `bash run.sh run` is only a small example, not the full test suite.

Problem and required behaviour

A scripted provider emits one command per invocation. The runner accepts READ, LIST, PROPOSE and FINISH. READ must precede LIST; both must precede PROPOSE. Invalid or out-of-order commands consume an attempt but do not authorise a mutation. PROPOSE returns AWAITING_APPROVAL. No operation in this exercise moves a booking.

Task 1: Implement bounded coordination

Complete Runner.run with a limit of 1–6 provider calls inclusive. Count every invocation, including invalid output. Return STEP_LIMIT on exhaustion and PROVIDER_ERROR on a provider exception.

Task 2: Enforce ordering and scope

Track read/list state locally to one run. Reject PROPOSE before the required observations. FINISH returns NO_ACTION. Unknown commands do not execute arbitrary code.

Task 3: Test the trace

Use a provider with a call counter. Test a valid three-command run, invalid outputs, early proposal, early finish and a thrown exception. Record both result and calls.

Cases to demonstrate

Case	Input or situation	Expected observation
1	READ, LIST, PROPOSE with limit 6	AWAITING_APPROVAL; 3 calls
2	Six UNKNOWN commands	STEP_LIMIT; exactly 6 calls
3	PROPOSE, READ, LIST, PROPOSE	First rejected; final proposal after 4 calls
4	FINISH	NO_ACTION; 1 call
5	Provider throws	PROVIDER_ERROR; no mutation
6	limit 0 or 7	IllegalArgumentException

The table defines required behaviour. Include fresh-state setup and actual expected results in tests. Do not weaken the conditions or delete a check to obtain a green result. You may add helper types and improve internal names, but preserve the public operations used by the starter checks or document a compatible adapter.

Working sequence

Use approximately 8 minutes for opening/compiling and reading, 12 for a design or trace on paper, 35 for the main change and focused tests, 15 for negative cases and explanation, and 10 for submission and feedback. The mandatory tasks fit the provided scope; extensions are optional and ungraded.

Use no generative AI for the initial trace and design explanation. During implementation you may use designated AI assistance only if you record the affected files and verification in `AI_USE.md`; you must be able to explain the resulting code. The supervised lab tests have different rules and prohibit generative AI.

Submission/check-out

By the end of your registered session, submit `Lab04_STUDENTNUMBER.zip` to the corresponding eClass practice activity, or show it to the TA if the practice upload is unavailable. Include `src/`, `test/`, `README.md`, `AI_USE.md`, and `results.txt`; omit generated classes, downloaded libraries and credentials. In `README.md`, identify each completed task, one limitation, and the exact command used. Include any requested diagram as `diagram.pdf` or `diagram.png` and a text alternative.

Run `bash run.sh check` and `bash run.sh test` when the cached JUnit dependency is available. Save their actual output. The included Jupiter wrapper runs the supplied checks; add focused Jupiter test methods of your own. A wrapper alone is not sufficient evidence of independent test design. If setup fails, preserve the exact error and continue with the plain-Java checks; do not label that output JUnit.

At check-out, explain one line of your implementation and one negative case without AI assistance. The TA gives formative feedback using the criteria below; there is no extra attendance grade.

Feedback criteria

Correct required behaviour (40%), useful tests including rejection/state checks (25%), clear responsibilities and code explanation (20%), and reproducible files plus attribution (15%). These percentages structure feedback only; they do not add a new assessment to the syllabus.

Optional extension

Add one additional failure case and explain what it reveals. Do not begin an unrelated frontend or live integration.